

Case Study Problem Statement: Gym Membership Management System

Objective:

Design and implement a Gym Membership Management System using Java and Object-Oriented Programming (OOP) principles. The system should allow gym administrators to manage members, assign membership plans, and view member information in a structured and maintainable way.

Problem Description:

Your task is to build a console-based application that will help a gym manage its members and membership plans. The application should support the following functionalities:

Add New Gym Members

- Collect member ID, name, and age.

- Store member details for future reference.

Assign a Membership Plan to a Member

- Available plans: Basic, Premium, and Gold.

- Each plan has a name, duration (in months), and fee.

- The system should allow assigning one of these predefined plans to a member.

View All Registered Members and Their Details

- Show member ID, name, age, and assigned membership plan details.

- If no plan is assigned, indicate it clearly.

Exit the System

- Option to gracefully exit the program.

Technical Requirements:

You must implement the system using Object-Oriented Programming in Java and demonstrate the following OOP concepts:

OOP Concept Requirement

Encapsulation

- Use private fields and provide public methods for access.

Inheritance

- Create a base abstract class for common attributes (e.g., Person).

Abstraction

- Use an abstract class that cannot be instantiated but can be extended.

Polymorphism

Implement overridden methods (e.g., showDetails()) to display user-specific information.

Collections

Use Java's ArrayList to store dynamic lists of members and plans.

Implement Exception Handling

Expected Classes and Structure:

Person – Abstract class with basic attributes like name and age.

Member – Extends Person, adds memberId and a MembershipPlan reference.

MembershipPlan – Class containing planName, durationMonths, and fee.

Gym – Manages the lists of members and plans.

Main – Contains the main method and menu-based user interaction.

Bonus Challenges (Optional):

Add input validation and error handling.

Save and load data from a file.

Allow updating or deleting member records.

Create a graphical interface using Swing or JavaFX.

Evaluation Criteria:

Use of all major OOP concepts

Code readability, reusability, and modularity

Use of collections and control structures

Proper use of constructors, methods, access modifiers and exception handling